SOFTWARE NOIL

Manual técnico

Noil Web Template

Cómo levantar un sistema Noil desde cero — Software NOIL

Qué trae la plantilla base de los sistemas web de Noil, cómo se crea un proyecto nuevo a partir de ella, qué hay que configurar paso a paso (base maestra `admin_tecnica`, producto, llave AES, conexiones dinámicas por cliente) y cómo se trabaja dentro de ella: estructura, servicios, componentes, reglas y cierre de cada tarea. Para el equipo de desarrollo de DSAIX.

Versión	Estatus	Notas
1.0.0 — 01/10/2026	Vigente	Plantilla con login contra `admin_tecnica`, tenant dinámico y sincronización de catálogos.

Control de versiones

Versión	Fecha	Autor	Cambios
1.0.0	01/10/2026	Equipo Noil	• Versión inicial del manual.

1. Qué es la plantilla

Es el punto de partida de todo sistema web nuevo de Noil. Trae resuelto lo que antes se copiaba de un repo a otro: el diseño, los componentes, el login contra Admin Técnica, la conexión dinámica a la base de cada cliente, los servicios de backend que usan todos los sistemas y las reglas de trabajo para personas y agentes de IA.

```repositorio
https://github.com/DSAI2017/noil_web_template
```

Stack

Capa	Tecnología
Backend	Laravel 13 sobre PHP 8.3 o superior. Fortify para autenticación, Wayfinder para rutas tipadas.
Frontend	Inertia v3 + React 19 + TypeScript. Tailwind v4, shadcn/Radix, iconos lucide.
Base de datos	MariaDB/MySQL en producción (maestra `admin_tecnica` + una base por cliente). SQLite solo para desarrollo y pruebas.
Herramientas	Composer, npm (Node 22), Pint, PHPStan, Vite (`vp`), PHPUnit.

Qué incluye

- **Tema Noil** en `resources/css/app.css`: paleta, Plus Jakarta Sans, sidebar oscuro, badges de estatus. Solo modo claro.
- **Sistema de diseño** en `resources/js/components/noil/`: `PageShell`, `ContextPanel`, `DataTable`, `EmptyState`, `confirmDialog`, `FlashToaster`, gráficas.
- **Login contra Admin Técnica**, selección de cliente y sincronización de catálogos a la base del cliente (sección 5).
- **Servicios de backend**: `MaximoId`, `AutoFolio`, `DynamicQueryExecutor`, `Kardex`, `PasswordAes`, `TenantConnection`, `AdminTecnicaSync`. Con tests.
- **Páginas demo**: `/dashboard` y `/ejemplo` (listado + drawer). Son la referencia para copiar.
- **Validación en español** (`lang/es`).
- **Skills** en `.agents/skills/`: reglas que cargan Claude Code y Codex antes de escribir código.

2. Requisitos en la máquina

Herramienta	Versión y notas
PHP	8.3 o superior con extensiones `pdo_mysql`, `pdo_sqlite`, `openssl`, `mbstring`, `intl` (esta última la piden algunos comandos de Artisan como `db:show`).
Composer	2.x.
Node y npm	Node 22 LTS.
Base de datos	Acceso a la `admin_tecnica` del ambiente (desarrollo o producción) y a la base del cliente con el que vas a probar. Para arrancar sin nada de eso, SQLite basta (sección 3).
Docker	Opcional. Solo para compilar manuales como este (`docs/_plantillas/compilar.sh`).

3. Crear un proyecto desde cero

Paso 1. Clonar la plantilla con el nombre del proyecto

```terminal
git clone https://github.com/DSAI2017/noil_web_template appweb_mi_sistema
cd appweb_mi_sistema
git remote rename origin plantilla
git remote add origin https://github.com/DSAI2017/appweb_mi_sistema
git push -u origin main
```

Dejar el remoto `plantilla` permite traer mejoras futuras con `git pull plantilla main`. Si prefieres partir limpio, borra la carpeta `.git` y haz `git init`.

Paso 2. Instalar dependencias y ambiente

```terminal
composer install
npm install
cp .env.example .env
php artisan key:generate
```

Paso 3. Arrancar en modo demo (SQLite)

El `.env.example` viene con `DB_CONNECTION=sqlite` y `NOIL_SYNC=false`. Así la plantilla levanta sin ninguna base externa: la migración crea las tablas mínimas de Admin Técnica y el seeder mete un usuario y un cliente demo.

```terminal
php artisan migrate:fresh --seed
composer run dev
```

Entra en `http://localhost:8000` con `admin@noil.mx` / `password`. `composer run dev` levanta servidor, Vite, cola y logs en una sola terminal.

Para qué sirve el modo demo

Para ver la plantilla funcionando, desarrollar pantallas que todavía no dependen de datos reales y correr las pruebas. Ningún sistema se despliega así: el siguiente paso es apuntarlo a la `admin_tecnica` real.

Paso 4. Conectar a Admin Técnica y al producto

Edita el `.env` con los datos del ambiente. Son las únicas variables propias de Noil:

```env
APP_NAME="Mi Sistema"
APP_URL=http://localhost:8000

DB_CONNECTION=mariadb
DB_HOST=<host de admin_tecnica>
DB_PORT=3306
DB_DATABASE=admin_tecnica
DB_USERNAME=<usuario>
DB_PASSWORD=<contrasena>

NOIL_PRODUCTO_ID=<IdProducto en master_productos>
NOIL_AES_KEY=<llave AES de admin_usuarios.SPassword>
NOIL_SYNC=true
```

Variable	Qué es y de dónde sale
`DB_CONNECTION` y `DB_*`	La conexión **maestra**. Siempre apunta a `admin_tecnica`; la plantilla la usa para validar el login y leer los clientes.
`NOIL_PRODUCTO_ID`	El `IdProducto` de este sistema en `master_productos`. Decide qué módulos, opciones y roles se sincronizan a la base del cliente.
`NOIL_AES_KEY`	La llave con la que Admin Técnica cifra `SPassword` (`AES_ENCRYPT`). Se pide al responsable de `admin_tecnica`. **Nunca** se versiona.
`NOIL_SYNC`	`true` en cualquier ambiente con bases de cliente reales. `false` solo en el modo demo con SQLite.

La llave AES no va al repositorio

`.env.example` la deja vacía y `config/noil.php` no tiene valor por defecto. Si la llave aparece en un commit, hay que rotarla en Admin Técnica, no basta con borrarla del historial.

Después de cambiar el `.env`:

```terminal
php artisan config:clear
php artisan migrate
```

Contra una `admin_tecnica` real la migración de tablas de Admin Técnica no hace nada (está guardada con `Schema::hasTable`); solo crea `sessions`, `cache` y `jobs` si no existen.

Paso 5. Darle identidad al proyecto

- `APP_NAME` en `.env`: aparece en el título de las páginas y en el sidebar.
- Logo en `resources/js/components/layout/app-logo-icon.tsx` e imágenes en `public/images/` (la portada del login es `login-cover.webp`).
- Menú en `resources/js/components/layout/app-sidebar.tsx` (`mainNavItems`).

- Borra el módulo de ejemplo cuando ya tengas el primero real: `EjemploController`, sus rutas, `pages/ejemplo`, `components/ejemplo`, `hooks/use-ejemplo-*` y `tests/Feature/EjemploTest.php`.
- Actualiza el `README.md` con el nombre y propósito del sistema.

4. Estructura del proyecto

árbol

```
app/
  Domain/<Modulo>/        modelo, interfaz, repositorios, casos de uso
  Http/Controllers/       controladores delgados (+ Settings/, TenantController)
  Http/Middleware/        HandleInertiaRequests, SetTenantConnection
  Http/Requests/          Form Requests con atributos en espanol
  Models/User.php         usuario de admin_usuarios (conexion maestra)
  Providers/              AppServiceProvider, FortifyServiceProvider
  Services/               MaximoId, AutoFolio, DynamicQueryExecutor, Kardex,
                          PasswordAes, TenantConnection, AdminTecnicaSync
config/noil.php           producto_id, master_connection, aes_key, sincronizar
database/migrations/      sessions, cache, jobs, admin_tecnica (guardada)
database/seeders/         usuario y cliente demo
resources/js/
  components/             ui/ noil/ layout/ auth/ <modulo>/
  hooks/                  use-<modulo>-<cosa>.ts
  layouts/                app (sidebar), auth, settings
  lib/                    fechas, utils, datos de muestra
  pages/                  una carpeta por modulo (+ tenant/, settings/)
  types/                  tipos compartidos
  actions/ routes/        generados por Wayfinder (no se editan)
.agents/skills/           ui-noil, backend-noil, flujo-noil
docs/_plantillas/         kit LaTeX para manuales como este
```

5. Login, cliente activo y sincronización

Es la parte que hace multi-cliente a los sistemas Noil. Está repartida en piezas pequeñas que no se conocen entre sí.

Flujo

1. El usuario escribe correo y contraseña en `/login`.
2. `FortifyServiceProvider` busca el correo en `admin_usuarios` (conexión maestra), verifica que esté activo, que la contraseña coincida con `PasswordAes` y que tenga al menos un cliente en `admin_clientes_roles_usuarios`. Si algo falla, el error es “credenciales incorrectas”.
3. En la siguiente petición, el middleware `SetTenantConnection` ve que no hay cliente en sesión:
   - Un solo cliente: lo selecciona, sincroniza y sigue.
   - Varios: redirige a `/cliente` (`pages/tenant/seleccionar.tsx`) para que elija.

4. Seleccionar un cliente (`TenantConnection::seleccionar`) hace tres cosas: configura la conexión `cliente_{IdCliente}` y la deja como conexión por defecto, guarda `tenant_id` en sesión y corre `AdminTecnicaSync` si `NOIL_SYNC` está activo.
5. A partir de ahí, cada petición restaura la conexión del cliente. Todo lo que use `DB::table()` o un modelo sin conexión explícita cae en la base del cliente. El usuario puede cambiar de cliente desde su menú.

Piezas

Pieza	Responsabilidad
`App\Models\User`	Tabla `admin_usuarios`, llave `IdUsuario`. Siempre en la conexión maestra: no sigue al tenant. Expone `name` y `email` como alias de `SNombre` y `SMail` para el frontend.
`PasswordAes`	`AES_ENCRYPT`/`AES_DECRYPT` de MySQL en PHP (AES-128-ECB con la llave plegada a 16 bytes). Valida sin mandar la llave a la base.
`TenantConnection`	Único lugar que conoce las credenciales de `admin_clientes`. Métodos: `usar`, `cliente`, `clientesDe`, `actual`, `seleccionar`. La sesión guarda solo el `IdCliente`.
`AdminTecnicaSync`	Espejea en la base del cliente los catálogos del producto: `admin_clientes`, `master_productos`, `master_idioma`, `admin_clientes_roles`, `admin_usuarios`, `master_modulos` (destino `master_modulos_admin`), `master_opciones`, `admin_roles_modulos`, `admin_clientes_roles_usuarios`. Borra e inserta por bloques de 500 dentro de una transacción, con un lock por cliente para que dos logins simultáneos no se pisen.
`SetTenantConnection`	Middleware del grupo web. Restaura la conexión en cada petición y resuelve el primer ingreso.
`TenantController`	`GET /cliente` muestra los clientes del usuario; `POST /cliente/\{id\}` lo selecciona (rechaza con 403 un cliente ajeno).

Lo que la plantilla ya no trae

Registro, recuperar o cambiar contraseña, borrar cuenta, 2FA y passkeys. Las cuentas se administran desde Admin Técnica. Queda el perfil (nombre y correo) y la pantalla de apariencia.

sContrato no lo pone el login

El servicio `Kardex` toma `sContrato` de la sesión. El contrato es por proyecto, no por cliente, así que el módulo que elija proyecto debe dejarlo con `session(['sContrato' => ...])`. Hasta entonces el kardex escribe con contrato vacío.

6. Servicios de backend

Todos reciben una conexión opcional; sin ella usan la conexión activa (la del cliente).

Necesito	Usar
Siguiente ID de una tabla	`(new MaximoId)->getIdMaximo($contrato, 'tabla', 'IdCampo')`. Nunca `MAX(Id)+1` a mano.
Folio con prefijo y consecutivo	`(new AutoFolio)->generar('tabla')`, con `$idDepartamento` para series por departamento.
Bitácora del sistema	`$kardex->registrar('Puesto 7 [Gerente] registrado', 'Catalogo de Puestos')` en toda alta, edición o baja.
SQL guardado en `master_consultassql`	`(new DynamicQueryExecutor)->executeQueryByName('nombre', [...])` o la versión paginada. Nunca copiar el SQL al código.

Sus tests están en `tests/Feature/ServiciosNoilTest.php`; los del login y la sincronización en `AuthTest.php` y `AdminTecnicaSyncTest.php`.

7. Reglas de backend

- **Un módulo, una carpeta** en `app/Domain/<Modulo>/`: modelo (solo tipado), interfaz del repositorio, `DynamicQuery*Repository` (consultas nombradas, la vía principal), `Eloquent*Repository` (solo para prototipar) y caso de uso cuando hay reglas de negocio. El `bind` va en `AppServiceProvider`.
- **Controlador delgado**: Form Request, llamada al repositorio o caso de uso, respuesta. Sin `DB::` ni modelos directos.
- **Rutas con nombre** (`modulo.accion`) y después `php artisan wayfinder:generate --with-form`. No reutilizar nombres de Fortify (`login`, `logout`, `password.*`).
- **Columnas con prefijo Noil**: `IdCliente`, `SNombre`, `EActivo` (Si/No), `ICantidad`, `DFechaInicio`, `URL_Icono`. Nada de snake_case nuevo.
- **Validación** en Form Requests con `attributes()` en español. Los errores 422 los reparte Inertia por campo.
- **Respuestas de escritura**: `back()->with('success', ...)` o `->with('error', ...)`; el toast ya está montado.

8. Reglas de frontend

- **Sin colores a mano**: `bg-primary`, `text-muted-foreground`, `bg-success`, `bg-warning`, `bg-info`, `bg-destructive`. Sin `dark:`.
- **Listado** = `PageShell` + `ContextPanel` + `DataTable`. El paginador de Laravel entra directo. Búsqueda, exportar, columnas y paginación ya vienen.
- **Captura** en un `Sheet` derecho (`*-form-drawer.tsx`) con `Field`, `FieldLabel`, `FieldError`

y `FormErrorBanner`. Detalle en `Dialog`. Borrado con `confirmDialog`.

- **Fechas**: `DateInputPicker` para capturar, `formatDate` para mostrar (`dd-mm-aaaa`); viajan al backend como `aaaa-mm-dd`. Nunca `<input type="date">`.
- **Componentes sin lógica**: todo `useForm`, `router.*` y cálculo vive en `hooks/use-<modulo>-<cosa>.ts`. Máximo 200 líneas por archivo.
- **Rutas por Wayfinder** (`@/routes`, `@/actions`), nunca strings. Sin `fetch` ni axios: `Inertia::defer()` o `useHttp`.
- **Caché de prefetch**: `Link` del menú con `cacheTags`, y toda escritura con `invalidateCacheTags`.

9. Arrancar un módulo nuevo

Pasos, en orden, para un catálogo "Proveedores". Cada paso deja la app funcionando.

1. Consulta nombrada del listado en `master_consultassql`.
2. `app/Domain/Proveedor/`: modelo, interfaz, `DynamicQueryProveedorRepository` (o `Eloquent*` mientras no existe la consulta).
3. `php artisan make:request Proveedores/StoreProveedorRequest y Update...`.
4. `php artisan make:controller ProveedorController` con `index/store/update/destroy`.
5. Rutas `proveedores.*` en `routes/web.php` dentro del grupo `auth`; luego `wayfinder:generate`.
6. Hooks `use-proveedor-listado.ts` y `use-proveedor-form.ts` copiando los de `ejemplo`.
7. Columnas y badge en `components/proveedor/columnas.tsx`.
8. Página `pages/proveedores/index.tsx` copiando `pages/ejemplo/index.tsx`.
9. Drawer `components/proveedor/form-drawer.tsx`.
10. `NavItem` en `app-sidebar.tsx`.
11. `php artisan make:test ProveedorTest`: index con búsqueda, store con 422 y con éxito, destroy.

10. Skills para agentes de IA

Las reglas de las secciones 7, 8 y 9 viven en `.agents/skills/` y se cargan solas:

Skill	Cuándo se carga
`ui-noil`	Antes de tocar `resources/js`: qué componente usar, tema, drawers, tablas, hooks.
`backend-noil`	Antes de tocar `app/`, `routes/`, `database/`: servicios, dominio, consultas nombradas, columnas, validación, login y tenant.
`flujo-noil`	Al arrancar un módulo y antes de dar por terminada una tarea.

Claude Code las ve por los symlinks de `.claude/skills/`; Codex por `AGENTS.md`. Cuando una regla cambia, se edita la skill (es la fuente de verdad) y se versiona. Las guías de Laravel Boost se regeneran con `php artisan boost:update` y no se editan.

11. Comandos y cierre de cada tarea

Qué	Comando
Servidor + Vite + cola + logs	`composer run dev`
Pruebas	`php artisan test -compact`
Formato PHP	`vendor/bin/pint -dirty`
Lint y formato JS	`npx vp check -fix`
Tipos TypeScript	`npx tsc -noEmit`
Rutas tipadas tras cambiar rutas	`php artisan wayfinder:generate -with-form`
Build de producción	`npm run build`
Agregar un primitivo shadcn	`npx shadcn@latest add <nombre>`
Compilar un manual	`docs/_plantillas/compilar.sh archivo.tex`

Ninguna tarea se da por terminada sin esto en verde:

cierre obligatorio

```
vendor/bin/pint --dirty
php artisan test --compact
npx vp check --fix
npx tsc --noEmit
```

Si el cambio es visual, además se abre la pantalla y se revisa el estado vacío, un error de validación y el flujo completo de alta, edición y borrado.

12. Pruebas

- Corren sobre SQLite en memoria (`phpunit.xml`), con `NOIL_SYNC=true` y una llave AES de prueba.
- `tests/TestCase.php` fija la transacción de `RefreshDatabase` sobre la conexión maestra: un test que cambie de cliente no ensucia a los demás.

- Para probar algo que requiera un usuario con cliente, inserta el cliente y la relación y mockea `AdminTecnicaSync` (ver `AuthTest`). Para probar la sincronización real, `AdminTecnicaSyncTest` crea un SQLite temporal como base de cliente.
- `User::factory()` crea usuarios de `admin_usuarios` con contraseña `password`; `->inactivo()` los deja en `EActivo = No`.

13. Antes de desplegar

- `APP_ENV=production`, `APP_DEBUG=false`, `APP_URL` real.
- `DB_*` apuntando a la `admin_tecnica` del ambiente, `NOIL_PRODUCTO_ID` correcto, `NOIL_AES_KEY` cargada, `NOIL_SYNC=true`.
- `SESSION_DRIVER` y `CACHE_STORE` en `database` requieren que `sessions` y `cache` existan en la base maestra (`php artisan migrate`). El lock de sincronización usa el cache.
- `npm run build` y `php artisan config:cache`.
- El seeder no se corre en producción: los usuarios viven en Admin Técnica.
- Módulo de ejemplo borrado.

14. Problemas comunes

Síntoma	Causa y solución
"Unable to locate file in Vite manifest"	No hay build ni Vite corriendo. `npm run build` o `composer run dev`.
TypeScript no encuentra `@/routes/...` o `@/actions/...`	Falta regenerar Wayfinder tras cambiar rutas: `php artisan wayfinder:generate --with-form`.
Login dice credenciales incorrectas con datos buenos	Revisa `NOIL_AES_KEY`, que `EActivo` sea `Si` y que el usuario tenga fila en `admin_clientes_roles_usuarios`.
Error al seleccionar cliente: tabla `master_modulos_admin` no existe	La base del cliente no tiene las tablas espejo, o estás en SQLite con `NOIL_SYNC=true`. En demo ponlo en `false`.
Cambié el `.env` y no pasa nada	`php artisan config:clear`.
"The intl PHP extension is required"	Instala `php-intl`; lo piden algunos comandos de Artisan, no la app.
Residuos con dueño root al compilar un manual	El PDF lo genera Docker como root. `compilar.sh` limpia los auxiliares; si queda algo, `sudo rm`.

15. Este manual

Vive en `docs/_plantillas/manual-plantilla.tex` y se compila con Docker (imagen `texlive/texlive`) sin LaTeX instalado:

```terminal
cd docs/_plantillas
./compilar.sh manual-plantilla.tex
```

Para un manual nuevo, copia `plantilla_base.tex`, edita portada y cuerpo, y sigue el `README.md` de la carpeta. Cuando cambie algo de lo que aquí se describe, sube la versión en la portada y en el control de versiones.